

Undergraduate Course Paper, Jinan University

Exploration of TinyBERT Model Reproduction and Hybrid Function Improvement Based on Multi-Stage Distillation Learning

School of Intelligent Science and Engineering

Professional Artificial Intelligence

Name: Xue Tianxin

Student ID: 2022104328

July 2, 2025

Honesty Statement

I declare that the course paper submitted is the result of my independent research work. To the best of my knowledge, except where specifically noted in the text, the paper does not contain any previously published or written research results by others. I promise that all content in the paper is true and reliable.

Course Paper Author Signature: Xue Tianxin Signature Date: July 2, 2025

Exploring the Reproduction of TinyBERT Model Enhanced by Multi-Stage Distillation Learning and Improvements in Mixed Functions

[Abstract] TinyBERT is a lightweight model based on the BERT approach. Unlike other distillation methods that directly distill from a pre-trained teacher model during classification, it instead mimics the teacher's training process. The distillation process consists of two stages: General Distillation and Task-specific Distillation. The former imitates BERT's pre-training on large-scale corpora, while the latter focuses on classification training on specific tasks. Notably, in the pre-training distillation stage, the teacher model used is an un-finetuned BERT model that has only undergone pre-training; whereas in the task-specific classification distillation stage, the teacher model is a BERT model fine-tuned on the specific task. To improve TinyBERT's performance metrics, this paper initially explores a hybrid cls_loss function combining dynamic weights of teacher model consistency and actual label consistency. However, this approach has not yet achieved ideal results, indicating further research is needed to optimize the dynamic ratio between these components, thereby enhancing model performance while maintaining lightweight efficiency.

[TinyBERT, transformer, distillation]

Introduction

1.1 Research Background

Pre-trained language models (PLMs) require only unlabeled text and utilize context-independent word embeddings to initialize the first layer of the model, thereby enriching the training data. They can be widely applied in feature construction for target models and fine-tuning tasks. The BERT model [3], proposed in 2019, builds upon the transfer learning ideas from ELMo [1] and GPT [2] in the NLP domain. By combining the advantages of bidirectional context modeling and the transformer encoder architecture, BERT serves not only as a method for generating word embeddings but also as a powerful feature extraction model structure. It has proven highly adaptable to various NLP tasks and is now extensively used across numerous natural language processing fields.

Knowledge distillation techniques have been extensively studied in the field of natural language processing and pre-trained language models, laying the foundation for further optimization and performance enhancement.

1.2 Insufficient research status

Fine-tuning stable pre-trained language models on specific tasks, along with compressing large models—characterized by high parameter counts and demanding hardware requirements—through distillation methods to accelerate inference speed and reduce model size while maintaining accuracy, has become a necessity and prevailing trend in the lightweighting and industrialization of natural language processing models.

However, pre-trained language models often have a large number of parameters and long inference times, making them unsuitable for edge deployment. Additionally, some studies have shown that PLMs contain redundancy [6]. Therefore, balancing performance with parameter optimization based on BERT has become a research focus since 2020. However, the BERT model first learns from unsupervised text corpora and is then fine-tuned on specific task data, which increases the difficulty of knowledge distillation. As a result, an effective distillation method must be designed based on these two training stages.

1.3 Solution Approach

In the reproduced model TinyBERT [4], a novel Transformer knowledge distillation method is first proposed to extract embedding knowledge from BERT. By integrating three loss functions—embedding layer output loss, attention matrix loss, and prediction layer output loss—the approach more effectively captures information from the "teacher model" BERT.

Subsequently, for two-layer training, we propose a two-stage learning framework: distillation in the general stage and distillation in the task-specific stage. In the general distillation stage, an unfine-tuned BERT model serves as the teacher. After processing the general dataset based on this teacher model, TinyBERT mimics the behavior of the teacher network by applying the proposed Transformer distillation method on the general corpus, thereby obtaining a general-purpose TinyBERT model.

In the task-specific distillation phase, data augmentation is first performed on a specific dataset by combining GloVe word embeddings with BERT predictions, and then a BERT model fine-tuned on the specific task is used as the second teacher model. In the second stage, the student model is initialized with the general-purpose TinyBERT model trained in the general phase, and undergoes two-step distillation—first at the intermediate layers, then at the prediction layer—resulting in a final TinyBERT model that achieves both high speed and accuracy.

Upon re-experimentation and observation, it was found that although the loss continuously decreased during the specific distillation process of the prediction layer and the overall accuracy on the MRPC dataset reached 83%, the accuracy (acc) and F1 evaluation metrics fluctuated significantly throughout training. Therefore, drawing inspiration from relevant optimization literature [5], we attempted to modify the final layer's loss function to a hybrid cross-entropy loss combining the teacher model and actual labels, with their proportions dynamically adjusted during training—initially relying entirely on the teacher model's loss.

thus enabling rapid learning in the initial phase; then gradually increasing the proportion of real label loss to achieve more stable and optimized accuracy evaluation.

1.4 Main Contributions

The main contribution of the Tiny BERT model is:

(1) Propose a novel Transformer distillation method to effectively transfer the encoded linguistic knowledge from the teacher BERT model to TinyBERT.

(2) To address the two-stage training of BERT, we propose a two-stage distillation framework, further dividing the second stage into intermediate layer feature distillation and prediction layer distillation, ensuring that the student model TinyBERT can fully learn both general and task-specific knowledge.

(3) The reproduced TinyBERT4 model achieves over 96% of the performance of the BERT base model on GLUE tasks, while reducing the number of parameters by approximately 13.3% and decreasing inference time by about 10%.

This paper improves the mixed-loss Tiny BERT model by dynamically adjusting the combination ratio between the teacher model's consistency loss and the cross-entropy loss with true labels in cls_loss, enhancing the stability of accuracy and F1 performance during final prediction layer training. However, the overall effect is slight, and further practical tuning of the ratio and its dynamic variation function remains necessary.

2 Related Work

2.1 BERT Variant Research

In recent years, most variants of pre-trained language models have been built upon the Transformer attention mechanism, primarily consisting of two main sub-layers: multi-head attention and fully connected feed-forward networks, as shown in the following structure:

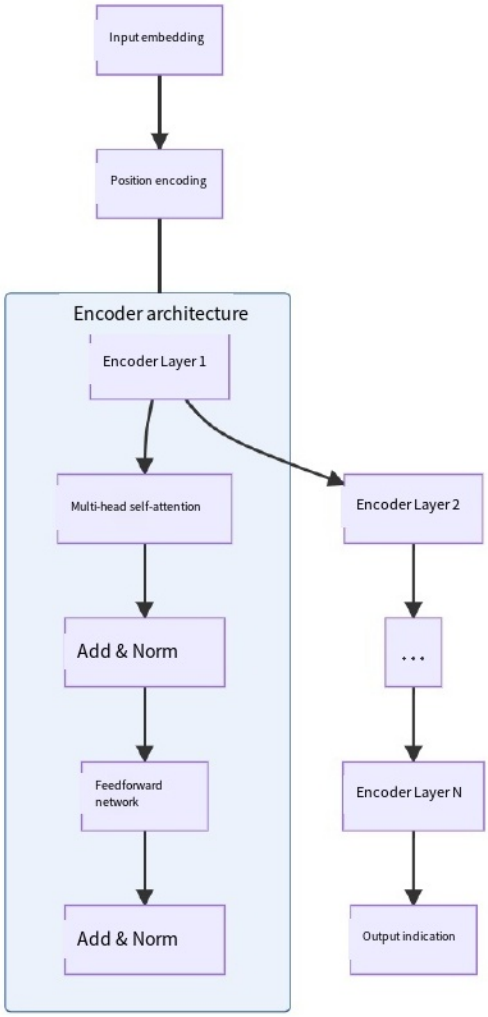


Figure 1 Attention Mechanism Framework

In comparative analysis of various variants, RoBERTa removes the NSP (Next Sentence Prediction) task and instead employs dynamic masking at each training epoch, along with longer training duration and larger batch sizes, achieving significant improvements over the original BERT on the GLUE benchmark dataset. However, it has drawbacks such as increased parameter count and longer training time. ALBERT, on the other hand, reduces the total number of model parameters substantially by sharing transformer parameters across layers, and enhances sentence-level understanding by replacing the NSP task with the SOP (Sentence Order Prediction) task.

DistilBERT and TinyBERT are both distilled models derived from the BERT base model, retaining 60% of the parameters from BERT-Base to achieve significantly faster inference speeds while preserving approximately 95% of the original performance. TinyBERT further refines this distillation process to achieve an even smaller model size, incorporating techniques such as intermediate layer alignment and attention matrix alignment. This makes it particularly suitable for resource-constrained environments, especially mobile deployment and online inference, offering a well-balanced trade-off between accuracy and efficiency.

3 Theoretical Framework

As shown in the figure, the TinyBERT training process consists of two parts: distillation from a general-purpose language model and task-specific distillation.

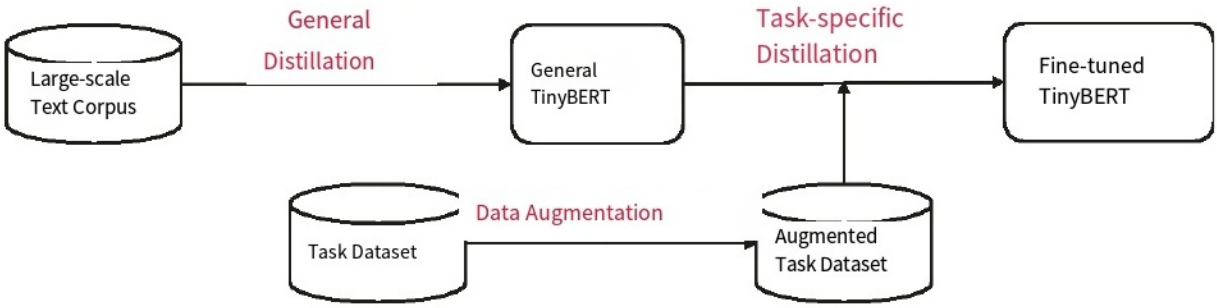


Figure 2 Overall Framework of TinyBERT

3.1 Transformer Knowledge Distillation

The BERT series learns through MLM and SEQ tasks, while TinyBERT employs distillation, which can be viewed as making the parameter distributions as close as possible. Many networks perform distillation only at the output layer, whereas TinyBERT conducts multiple rounds of distillation within the model's intermediate layers. Furthermore, distillation is applied repeatedly throughout the entire training process, enabling a more compact parameter size while comprehensively learning the teacher model's processing strategies at each stage.

As shown in the figure, the distillation of the transformer layer includes two distillation modules:

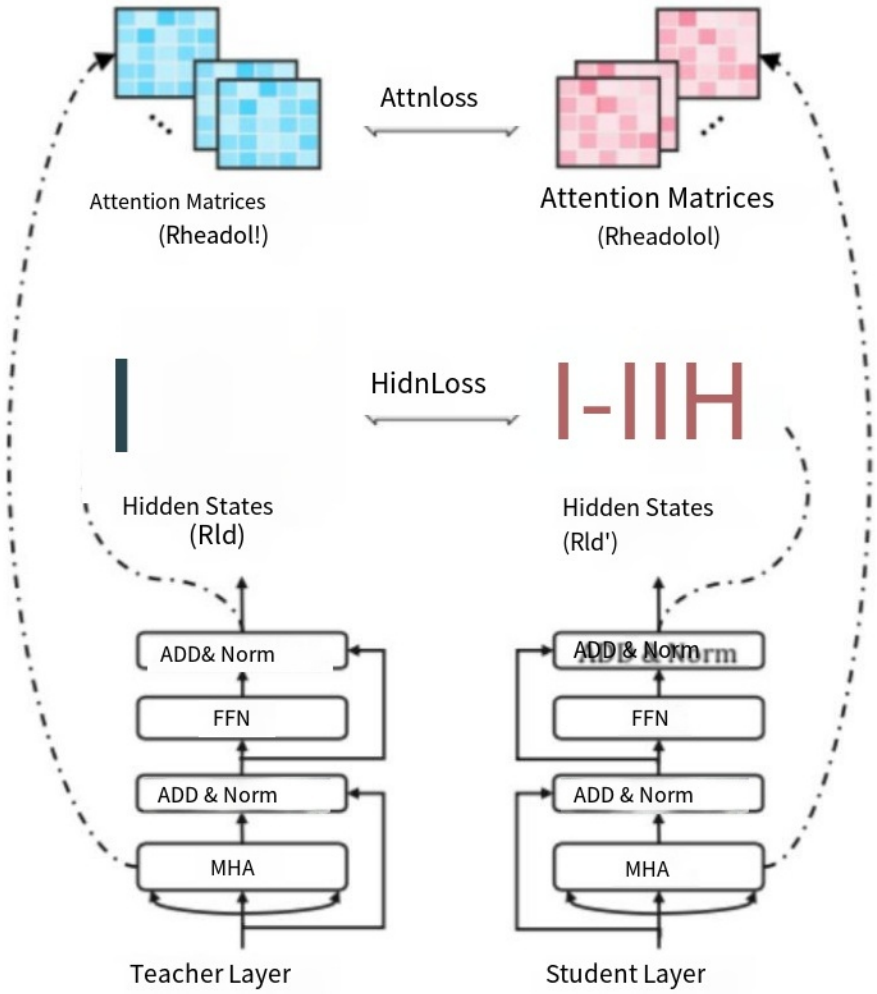


Figure 3: Analysis of the Transformer Distillation Architecture

(1) Attention-based distillation: Distill the attention mechanism matrix by training the corresponding student Encoder layer to minimize the mean squared error between the attention matrices of the student and teacher Encoders.

Its loss function is as follows:

$$\mathcal{L}_{attn} = \frac{1}{h}\sum_{i=1}^{h} MSE(A_i^S, A_i^T)$$

where h is the number of heads in the multi-head attention mechanism, and A_i^S and A_i^T are the attention matrices of the student model and the teacher model, respectively.

(2) Hidden state-based distillation: distilling hidden layer states, with the distillation loss function defined as:

$$\mathcal{L}_{hidn} = MSE(H^S W_h, H^T)$$

(3) Embedding-layer Distillation: Distilling the embedding layer, with the loss function being

$$\mathcal{L}_{embd} = MSE(E^S W_e, E^T)$$

where E represents the embedding layer vector, and W is a learnable parameter designed to map the student model's embedding vectors into the same high-dimensional space as those of the teacher model.

(4) Prediction-layer Distillation: In the prediction-layer distillation, knowledge information from the output layer is transferred. A loss function is employed

$$\mathcal{L}_{pred} = CE(z^T/t, z^S/t)$$

where z represents the model's output logits and t denotes the distillation temperature. The overall loss function is obtained by combining these four components. Essentially, this involves using the softmax temperature function to obtain corresponding soft targets and soft predictions, which are then described using the cross-entropy loss function.

3.2 Two-stage distillation

TinyBERT employs a two-stage distillation approach.

General Distillation: General TinyBERT is obtained by distilling the original BERT model as a teacher model on a large unlabeled text corpus.

Task-specific Distillation: A downstream task dataset is constructed through data augmentation. The general TinyBERT model is distilled onto the augmented dataset using a fine-tuned BERT, resulting in a specialized TinyBERT model—effectively treating the general TinyBERT as the initial model for the second distillation stage. Specifically, the data augmentation method employed in the paper involves first predicting masked words with BERT/GloVe, then replacing the masked positions with the most similar words, and randomly incorporating these enhanced examples into the dataset.

The specific algorithm is as follows:

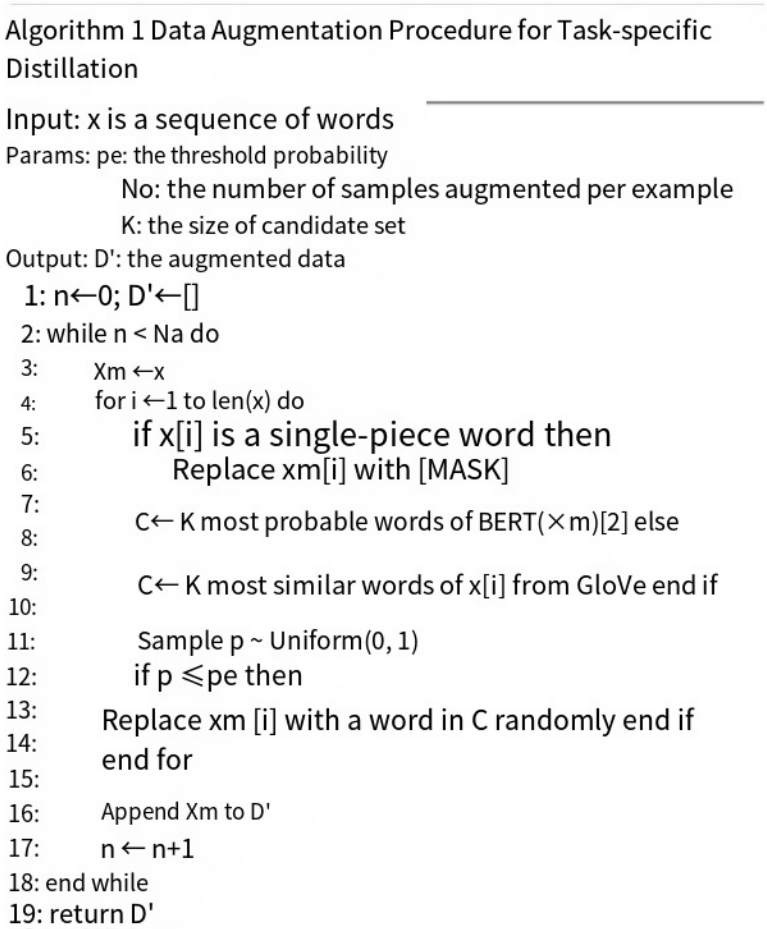


Figure 4: Two-stage distillation pseudocode framework

General Distillation: In this stage, a BERT-base pre-trained model is used as the teacher, and the student model is distilled using the BERT-base training set. This model is referred to as the General TinyBERT model.

Task-specific Distillation: During the fine-tuning phase, a general TinyBERT model is fine-tuned on a specific task. The fine-tuning process involves first fine-tuning a pre-trained BERT-base model on the target task to serve as the teacher model. Then, the general TinyBERT model obtained through general distillation acts as the student model. After distillation, the resulting TinyBERT model is referred to as the fine-tuned TinyBERT model.

4 Experimental Process

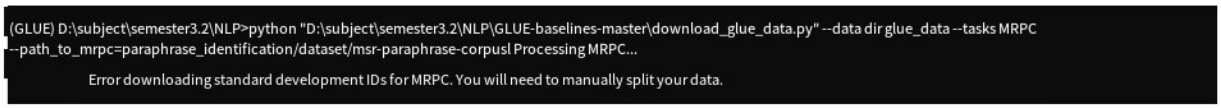
4.1 Dataset Preparation

(1) GLUE-MRPC Dataset Preparation

The GLUE dataset is a benchmark for evaluating natural language understanding (NLU) models, comprising nine tasks covering text similarity, sentiment analysis, and natural language inference. The MRPC dataset (Microsoft Research Paraphrase Corpus) is the semantic equivalence task within the GLUE benchmark, containing 3,665 sentence pairs from news corpora—3,681 pairs in the training set, 408 in the validation set, and 1,725 in the test set. It assesses a model's ability to determine whether two sentences express the same meaning. Despite its relatively small size, it presents significant challenges and effectively evaluates a model's generalization performance on sentence-pair tasks. Accuracy and F1 score are primarily used as evaluation metrics, making it a classic binary classification task for testing semantic similarity capabilities in natural language processing.

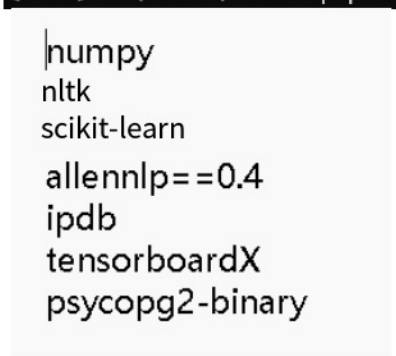
Download and refer to the nyu-mll/GLUE-baselines: [DEPRECATED] Repository for exploring multi-task learning approaches to learning sentence representations. The GLUE dataset requires a dedicated environment with pytorch=0.4.1 for downloading. Here, we create a Python 3.6 virtual environment and configure it step by step. The specific download process is as follows:











After observing the preparation environment, downloading the MRPC dataset directly results in an error. In this case, you need to download dev_ids.tsv from https://github.com/MegEngine/Models/tree/master/official/nlp/bert/glue_data/MRPC and place the downloaded file into the MRPC directory within the data_dir used when downloading the dataset.

Set (the data_dir in the command to glue_data), then modify download_glue_data.py in glue_data as follows, and run it again to successfully download (glue_data\MRPC).

```
(GLUE) D:\subject\semester3.2\NLP>git config --global http.sslVerify false

(GLUE) D:\subject\semester3.2\NLP>git clone https://github.com/wasiahmad/paraphrase_identification.git

Cloning into 'paraphrase_identification'... remote: Enumerating objects: 68, done. remote: Total 68
(delta 0), reused 0 (delta 0), pack-reused 68 (from 1) Receiving objects: 100% (68/68), 98.20 MiB 6.31 MiB/s,
done. Resolving deltas: 100% (12/12), done. Updating files: 100% (45/45), done.
```

```
(GLUE) D:\subject\semester3.2\NLP>python "D:\subject\semester3.2\NLP\GLUE-baselines-master\download_glue_data.py" --data

_dir glue_data --tasks MRPC --path_to_mrpc=paraphrase_identification/dataset/msr-paraphrase-corpus Processing MRPC... Completed!
```

(2) KIWI Dataset Preparation

The KIWI dataset originates from Wikipedia, a widely used online encyclopedia with extensive multilingual text resources, making it highly suitable for natural language processing (NLP) tasks.

① **wikiextractor extraction**

Use the first part of the kiwi20250620 dataset as large-scale unlabeled text for general pretraining of the model. After downloading the dataset, extract and organize the text using WikiExtractor. Since WikiExtractor is not supported, use WSL to complete the data processing and then copy the results back to the local Windows directory (KIWI_data\kiwidata_wsl).

```
2025-06-21 19:37:07 Completed the article, template, media/files description, and main meta pages. In
multiple bz2 streams, each stream contains 100 pages.

enwiki-20250620-pages-articles-multistream1.xml-p1p41242.bz2 - 276.4 megabytes
```

config.py × download_glue_data.py

```
D:> subject>semester3.2> NLP>GLUE-baselines-master> download_glue_data.py> format_mrpc
57      def format_mrpc(data_dir, path_to_data):
84
85          # · try:
86          #       URLLIB urlretrieve(TASK2PATH["KRPC"], os.path.join(mrpc_dir, "dev_ids.tsv
87          # · except KeyError or urllib error.HTTPError:
            #       print("\tError downloading standard development IDs for MRPC. You will ne
89          #       return
```

```
xueeux@XUEEUX:/mnt/c/Windows/system32$ mkdir -p~/wiki_processing

xueeux@XUEEUx:/mnt/c/Windows/system32$ cd/wiki_processing xueeux@XUEEUX:/wiki_processing$ cp /mnt/d/subject/semester3.2/NLP/enwiki-20250620-pages-articles-multistreaml.
xml-p1p41242.bz2 . xueeux@XUEEUx:~/wiki processing3 pip3 install wikiextractor Defaulting to user installation because normal site-packages is not writeable Collecting wikiextractor

  Downloading wikiextractor-3.0.6-py3-none-any.whl(46 kB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 46.4/46.4 KB 422.1 kB/s eta 0:00:00
Installing collected packages: wikiextractor
WARNING: The scripts extractPage and wikiextractor are installed in '/hone/xueeux/.local/bin' which is not on PATH. Consider adding this directory to PATH or, if
you prefer to suppress this warning, use --no-warn-script-location. Successfully installed wikiextractor-3.0.6
xueeux@XUEEUX: /wiki_processing$ python3 -m wikiextractor.WikiExtractor
    --json
    - no-templates \
    -b 1M
    -o kiwidata
enwiki-20250620-pages-articles-multistreaml. xml-p1p41242. bz2 INFO: Starting page extraction frow enwiki-20250620-pages-articles-multistreaml.
xul-p1p41242. bz2.
INFO: Using 19 extract processes.
INFO: Finished 19-process extraction of 27133 articles in 34.7s (783.0 art/s)
xueeux@XUEEUX:/wiki_processing$ cp -r kiwidata /mnt/d/subject/semester3.2/NLP/kiwidata_wsl
xueeux@XUEEUX: /wiki_processing3
```

② Plain text information generation

Write a program, TinyBERT/extract_text.py, to consolidate all the texts extracted in the previous step into a plain text file wiki_text.txt, with one article per line, for generating a JSON file that meets the model training requirements. It is observed that this dataset contains a total of 37,700 texts.

```
PS D:\subject\semester3.2\NLP> python TinyBERT\extract_text.py
Generated plain text file: D:\subject\semester3.2\NLP\wiki_text.txt Number of
lines: 3770
```

③ Pre-training data generation — generating JSON-format corpus

Modify the file tinybert\pregenerate_training_data.py to make it compatible with Windows, then set the path to the teacher model, the original corpus path (wiki_data), and the output path for JSON data. Finally, run the code. This Python program uses the tokenizer from the specified BERT base model to tokenize the raw text, and during each epoch, it randomly resamples masked positions and sentence order to meet the diversity requirements of the MLM and NSP training datasets. Generating multiple rounds of data increases training diversity and helps prevent the model from overfitting to a single static dataset.

```
PS D:\subject\semester3.2\NLP> $env:BERT_BASE_DIR = "D:\subject\semester3.2\NLP\Bert-base"
PS D:\subject\semester3.2\NLP> $env:CORPUS_RAW="D:\subject\semester3.2\NLP\wiki_text.txt"
PS D:\subject\semester3.2\NLP> $env:CORPUS_JSON_DIR="D:\subject\semester3.2\NLP\tinybert\corpus_json"
```

```
PS D:\subject\semester3.2\NLP> python tinybert\pregenerate_training_data.py
>>      --train_corpus $env:CORPUS_RAW
>>      --bert_model $env:BERT_BASE_DIR
>>      -- reduce_memory
>>      --do_lower_case
>>      -epochs_to_generate 3
Response --output_dir $env:CORPUS_JSON_DIR
```

```
[01:03, 91.10 lines/s]e7/03/2025 22:11:26 - INFO main
01:05, 93.52 lines/s]e7/03/2025 22:11:28 - INFO main
01:08, 75.93 lines/s]e7/03/2025 22:11:31 - INFO main
01:11, 103.15 lines/s]07/03/2025 22:11:33 - INFO main
01:13, 99.28 lines/s] 07/03/2025 22:11:35-INFO main
01:15, 80.42 lines/s]e7/03/2025 22:11:37 - INFO main
01:17, 104.56 lines/s]07/03/2025 22:11:40 -INFO main
01:19, 110.63 lines/s]07/03/2025 22:11:42 -INFO main
01:22, 112.91 lines/s]07/03/2025 22:11:44-INFO main
01:24, 84.44 lines/s]e7/03/2025 22:11:47 - INFO main
01:27, 65.69 lines/s]07/03/2025 22:11:49 - INFO main
01:29, 81.43 lines/s]e7/03/2025 22:11:52 - INFO main
01:32, 93.79 lines/s]07/03/2025 22:11:54 - INFO main
01:35, 51.55 lines/s]07/03/2025 22:11:57 - INFO main
01:37, 114.28 lines/s]07/03/2025 22:11:59 - INFO main
01:40, 63.43 lines/s]07/03/2025 22:12:02 - INFO main
01:42, 83.31 lines/s]07/03/2025 22:12:04 - INFO main
01:44, 94.11 lines/s]07/03/2025 22:12:07 - INFO main
01:47, 94.32 lines/s]07/03/2025 22:12:09 - INFO main
01:49, 133.65 lines/s]07/03/2025 22:12:12 -INFO main
01:51, 92.44 lines/s]e7/03/2025 22:12:14 - INFO main
01:54, 83.78 lines/s]07/03/2025 22:12:16 - INFO main
01:56, 76.27 lines/s]07/03/2025 22:12:18 INFO
01:59, 89.38 lines/s]e7/e3/2025 22:12:21 - INFO main
02:01, 94.44 lines/s]e7/03/2025 22:12:23 - INFO main
loaded 5000 docs! [02:03, 89.27 lines/s]07/03/2025 22:12:26 - INFO main
loaded 5100 docs! [02:06, 78.71 lines/s]07/03/2025 22:12:28 - INFO main
loaded 5200 docs! Loading Dataset: 10594 lines [02:08, 82.99 lines/s]07/03/2025 22:12:31 - INFO main
loaded 5300 docs! Loading Dataset: 10789 lines [02:11, 72.87 lines/s]07/03/2025 22:12:33 - INFO main
loaded 5400 docs! Loading Dataset: 10996 lines [02:13, 72.63 lines/s]07/03/2025 22:12:36 - INFO main

Loading Dataset: 5197 lines Loading Dataset: 5395 lines Loading Dataset: 5599 lines Loading Dataset: 5787 lines Loading Dataset: 5997 lines Loading Dataset: 6191 lines Loading Dataset: 6392 lines Loading Dataset: 6589 lines Loading Dataset: 6791 lines Loading Dataset: 6993 lines Loading Dataset: 7195 lines Loading Dataset: 7391 lines Loading Dataset: 7598 lines Loading Dataset: 7793 lines Loading Dataset: 7999 lines Loading Dataset: 8191 lines Loading Dataset: 8395 lines Loading Dataset: 8591 lines Loading Dataset: 8799 lines Loading Dataset: 8983 lines Loading Dataset: 9195 lines Loading Dataset: 9391 lines Loading Dataset: 9593 lines Loading Dataset: 9787 lines Loading Dataset: 9996 lines Loading Dataset: 10192 lines Loading Dataset: 10391 lines

loaded 2600 docs!
loaded 2700 docs!
loaded 2800 docs!
loaded 2908 docs!
loaded 3000 docs!
loaded 3100 docs!
loaded 3200 docs!
loaded 3300 docs!
loaded 3400 docs!
loaded 350e docs!
loaded 3600 docs!
loaded 3700 docs!
loaded 3800 docs!
loaded 3900 docs!
loaded 4000 docs!
loaded 4100 docs!
loaded 4200 docs!
loaded 4300 docs!
loaded 4400 docs!
loaded 4500 docs!
loaded 4600 docs!
loaded 4700 docs!
loaded 480e docs!
loaded 4900 docs!
loaded 5500 docs!
```

After the operation is completed, the word segmentation in the article is as follows:

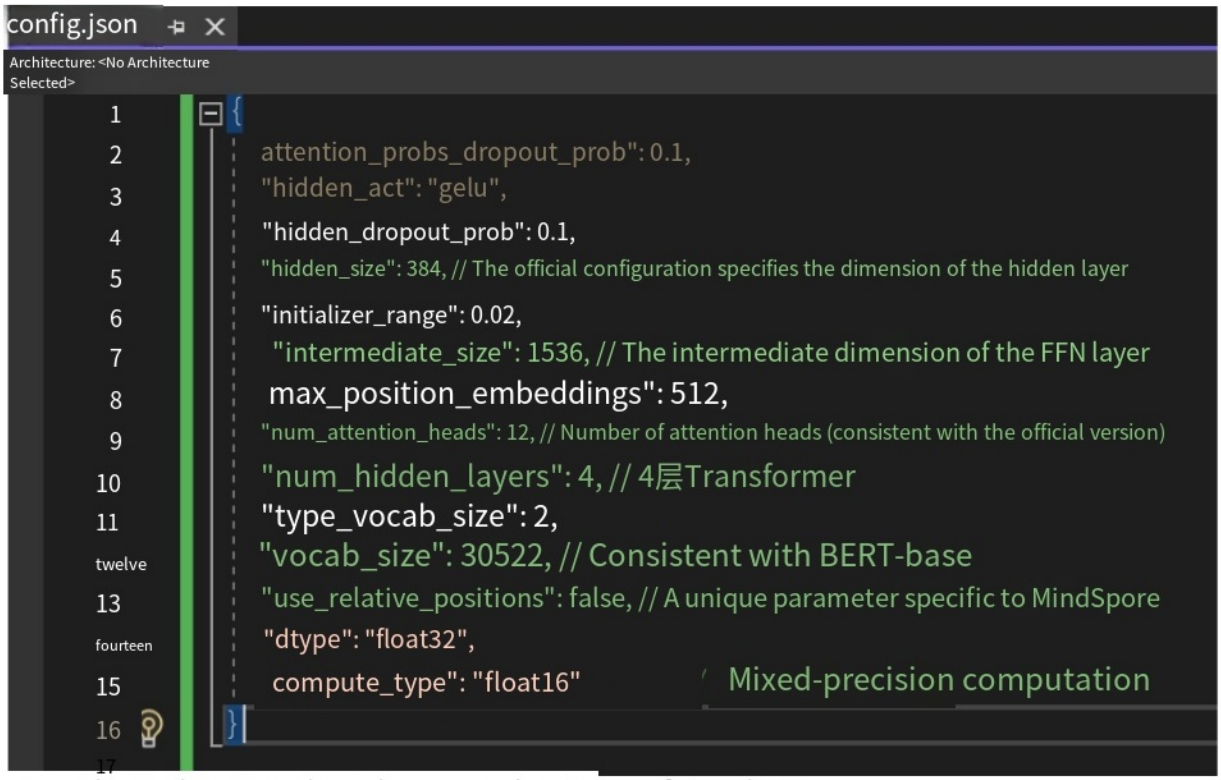


("tokens": ["[CLS]", "roman", "catholicism",",", "etc",".",")",",", "patriarch", "##y",".", "het", "##eros", "##ex","##ism",",", "white", "supremacy",",","and","imperialism","","", "anarchist", "schools", "disagree", "on", "the", "methods", "by", "which", "these", "forms", "should", "be", "opposed","", "tactics",".", "anarchist", "##s", "", "tactics", "take", "various", "forms", "but", "in", "general", "serve", "two", "major", "goals",",", "namely",",", "to", "first", "oppose", "the", "establishment", "and", "secondly","to", "[sEP]", "available","to","the", "islanders", "for", "cultivation",",", "encouraging", "the", "development", "of", "cattle", "raising",",", "import", "##ing", "deer", "and", "water", "buffalo", "from", "manila", ",","and", "donkey", "##s", "and", "mule", "##s", "from", "ac", "##ap","##ul", "##co", ",", "[MASK]", "cotton", "mills", "and", "salt", "pan","##s",",", "establishing", "free", "public", "schools",,", "and", "forming", "the", "first", "guam", "militia","", "he", "was", "transferred", "to", "manila", "in", "June", "[SEP]"], "segment ids"

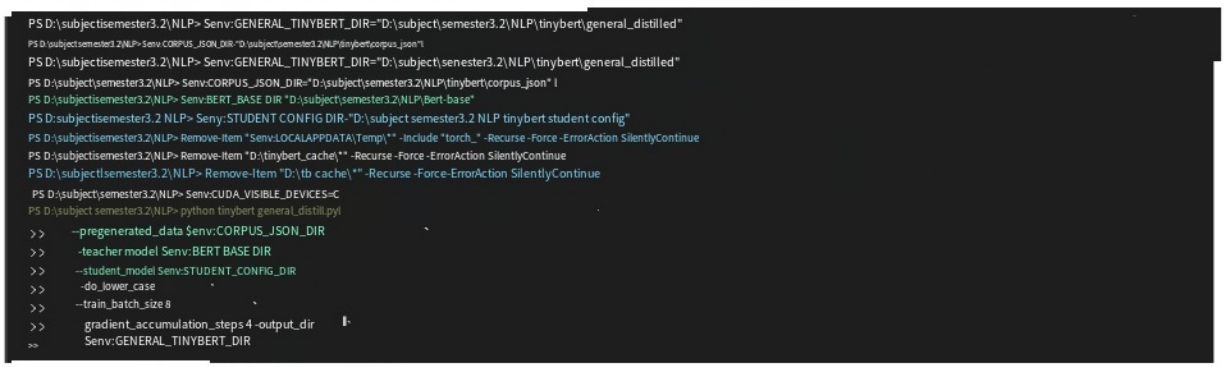
[0, 0, 0,

The configuration file for the student model, generated according to the settings in the official MindSpore version, is as follows:

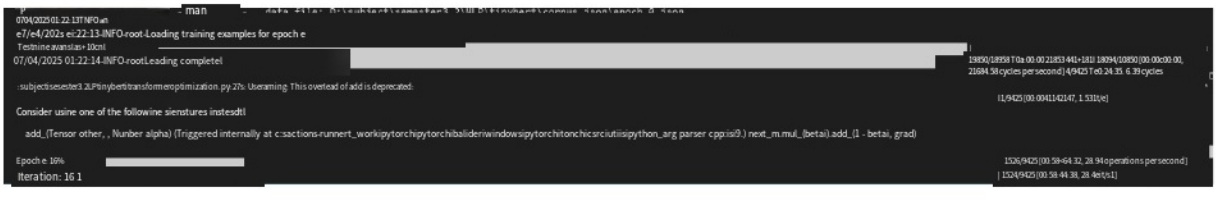
Modify the corresponding code according to the Windows and NumPy versions, and adjust the batch_size to 16 based on GPU memory requirements.



Run the code to conduct the general training of tiny_bert.

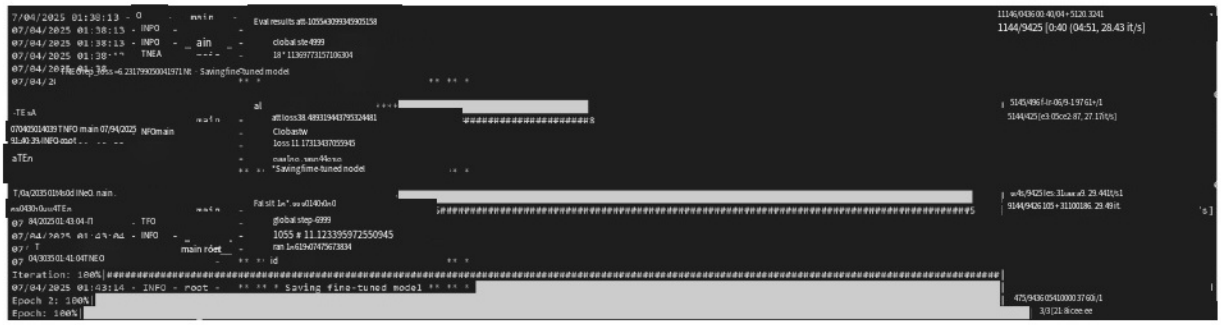


The first round is over.



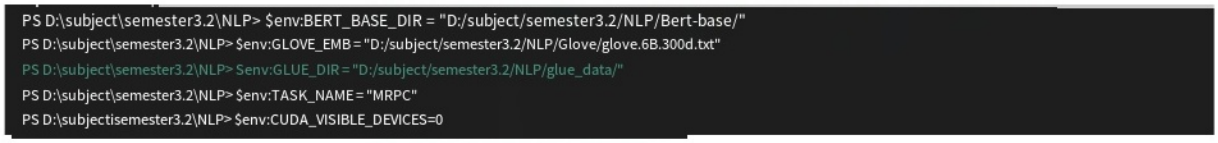


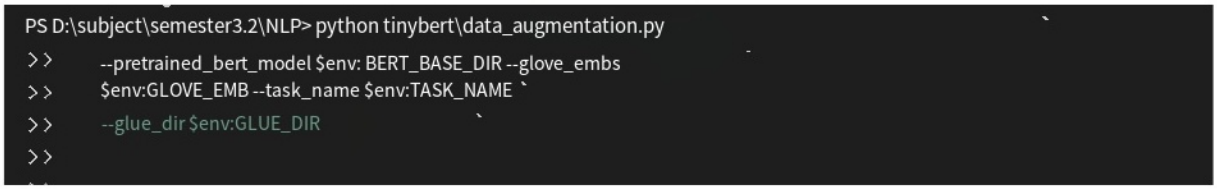
Round three ends:

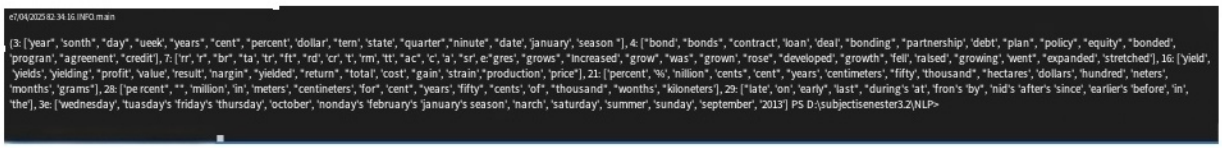


4.5 Data Augmentation

Data augmentation aims to expand the training set specifically for a given task, in this case, the MRPC semantic equivalence classification task. The data augmentation method used in this experiment combines BERT and GloVe for word-level replacement. For each word in a sentence, it jointly considers semantically similar words identified by GloVe and the most likely replacement candidates predicted by BERT, selecting the optimal substitute. This process ultimately generates new sentences that are semantically similar but lexically different.







Looking at any list in the log {1: ['nation', 'country', 'world', 'state', 'republic', 'people', 'territory', 'province', 'kingdom', 'region', ' island', 'colony', 'planet', 'city', 'government']}, it indicates that the program is extracting attention layer weights from the base BERT model. Currently, these are the top 15 most frequently attended words in the first attention layer. After execution, an augmented training set was observed to be generated in the glue\MRPC folder.

4.6 MRPC Distillation (Distillation for a Specific Task)

(1) Preparation of the bert_base model fine-tuned on MRPC



Download the BERT_base model fine-tuned for 5 epochs from huggingface_bert-base-uncased-MRPC. After downloading, set the subsequent distillation parameters.

```
PS D:\subject\semester3.2\NLP> Senv:GENERAL_TINYBERT_DIR = "D:/subject/semester3.2/NLP/tinybert/general_distilled/"
PS D:1subiect semester3.21NLP> Seny:TASK DIR = "D:/subiect/semester3.2/NLP/glue data/KRPC/"
PS D:\subject\semester3.21NLP> Seny:TMP TINYBERT DIR = "D:/subiect/semester3.2/NLP/tinybert/task distill intermediate/mrpc/"
PS D:\subject\semester3.2\NLP> $eny:TINYBERT_DIR = "D:/subject/semester3.2/NLP/tinybert/final_tinybert/mrpc/"
PS D:\subject\semester3.2\NLP> Senv:FT_BERT_BASE_DIR="D: \subject\semester3.2\NLP\huggingface_bert-base-uncased-MRPC"
```

(2) Middle distillation

```
PS D:\subject\semester3.2\NLP> python tinybert\task_distill py
>>      --teacher_model $env:FT_BERT_BASE_DIR
>>      - student_model $env:GENERAL_TINYBERT_DIR
>>      - data_dir $env:TASK_DIR
>>      - task_name MRPC
>>      - output_dir Senv:TMP_TINYBERT_DIR
>>      - max_seq_length 128
>>      - train_batch_size 8
>>      - num_train_epochs 5
>>      - aug_train
>>      - do_lower_case
>>      - gradient_accumulation_steps 4
>>      - eval_step 1000
```

```
PS D:\subject semester3.21NLP> python tinybertitask_distill.py
>>      --teacher_model Senv:FT_BERT_BASE_DIR
        --student_model Senv:GENERAL_TINYBERT_DIR
        --data_dir Senv:TASK_DIR
>>      --task_name MRPC
>>      --output_dir Senv:TMP_TINYBERT_DIR
>>      --max_seq_length 128                    -
>>                              --train_batch_size 8
>>                              --num_train_epochs 10
>>      --aug_train             --gradient_accumulation_steps 4
>>      --do_lower_case

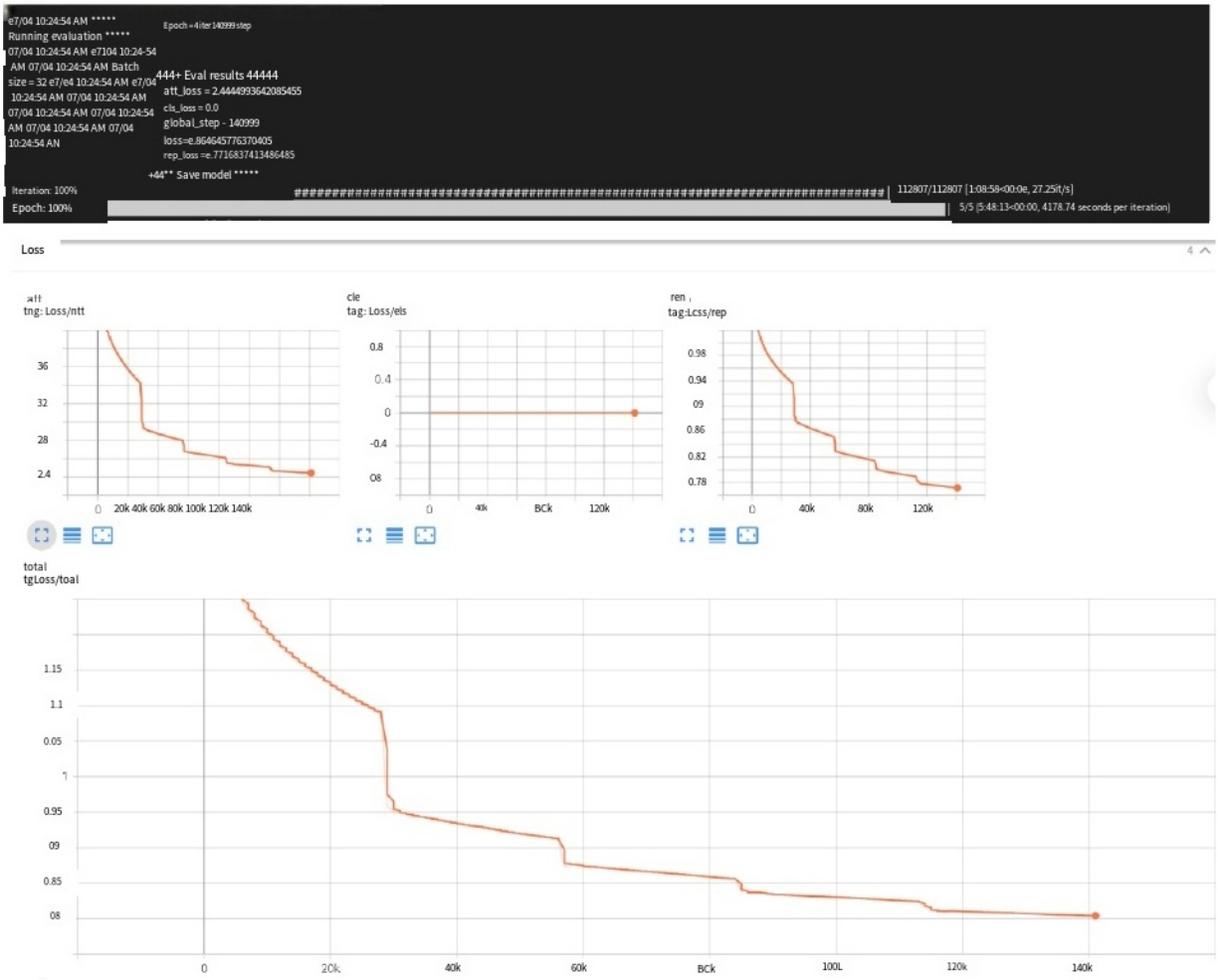
>>

e7/04 e4:16:43 AK The args: Namespace(data_dir='D:/subject/senester3.2/NLP/glue_data/KRPC/', teacher_nodel='D:11subjectiisenester3.211NLP\lhuggingface_bert-base-uncased-MRPC', student_nodel-'D:/subject/semester3.2/NLP/tinybert/general_distilled/, task_nawe-'MRPC',
output_dire'D:/subject/semester3.2/NLP/tinybert/task_distill_intermediate/nrpc/', cache dir='', max_seq_length-128, do_eval-False, do_lower case-True, train_batch_size-8, eval_batch_size-32, learning_rate-Se05, weight_decay=0.0001, nun_train_epochs=10.0,
warmup_proportion=e.1, no_cuda=False, seed=42, gradient_accunulation_steps=4, aug_train=True, eval_step=50, pr ed_distill=False, data_url='', temperature=1.8) 07/04 04:16:43 AM device: cuda n_gpu: 1 07/04 04:16:44 AK Nriting example e of 225613 07/04 04:16:44 AM ***
Example * 07/04 04:16:44 AM guid: aug-1 07/04 04:16:44 AM tokens: [CLS] " an ###ro ##zi accused his brother , whon he called " " the witness "", of deliberately di ##stor ##ting his evidence . " [s EP) "referring to hin as only ""the witness"",an ###ro ##zi accused his brother of
deliberately di ##stor ##ting his evidence . "[SEP) 07/04 04:16:44 AK input ids: 101 1000 2572 3217 5831 5496 2010 2567 1010 3183 2002 2170 1000 1000 1996 7409 1000 1000 1010 1997 9969 4487 23809 3436 2010 3350

1012 1000 102 1000 7727 2000 2032 2004 2069 1000 1000 1996 7409 1000 1000 1010 2572 3217 5831 5496 2010 2567 1997 9969 4487 23809 3436 2010 3350 1012 100
2,000,000,000,000,000,000,00
07/0404:16:44 AM input mask: 11111111111
```

After training, the observed changes in the loss function are as follows:

The observed attention loss eventually reached 2.44, which, although relatively high in absolute value, represents a 45% decrease from the initial value of 4.45. The final loss was 0.772, indicating that the student model has effectively reproduced the intermediate layer representations of the teacher model, demonstrating good distillation performance. The total loss converged to 0.804, reaching a low level.



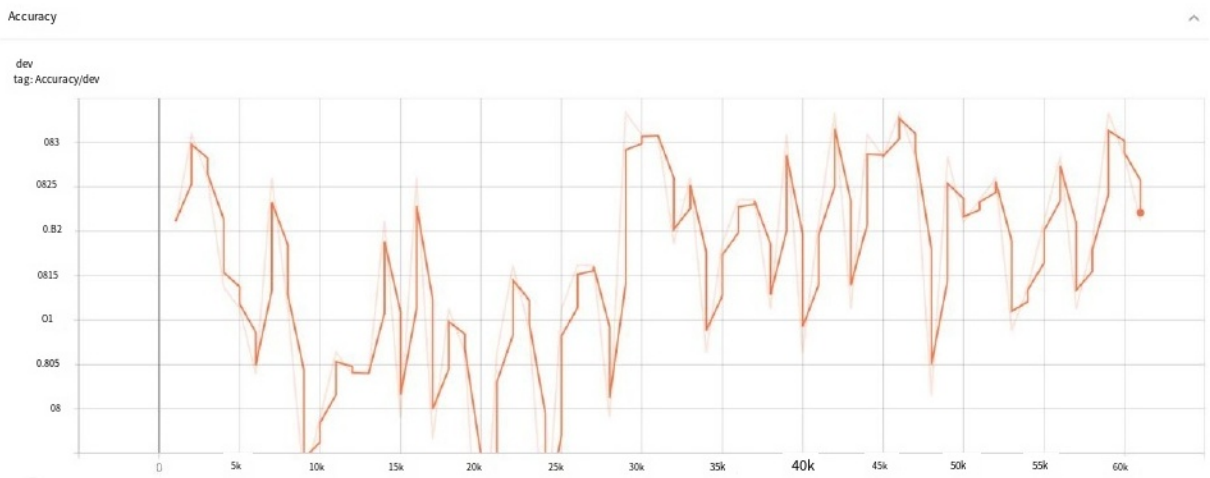
(3) Final layer distillation

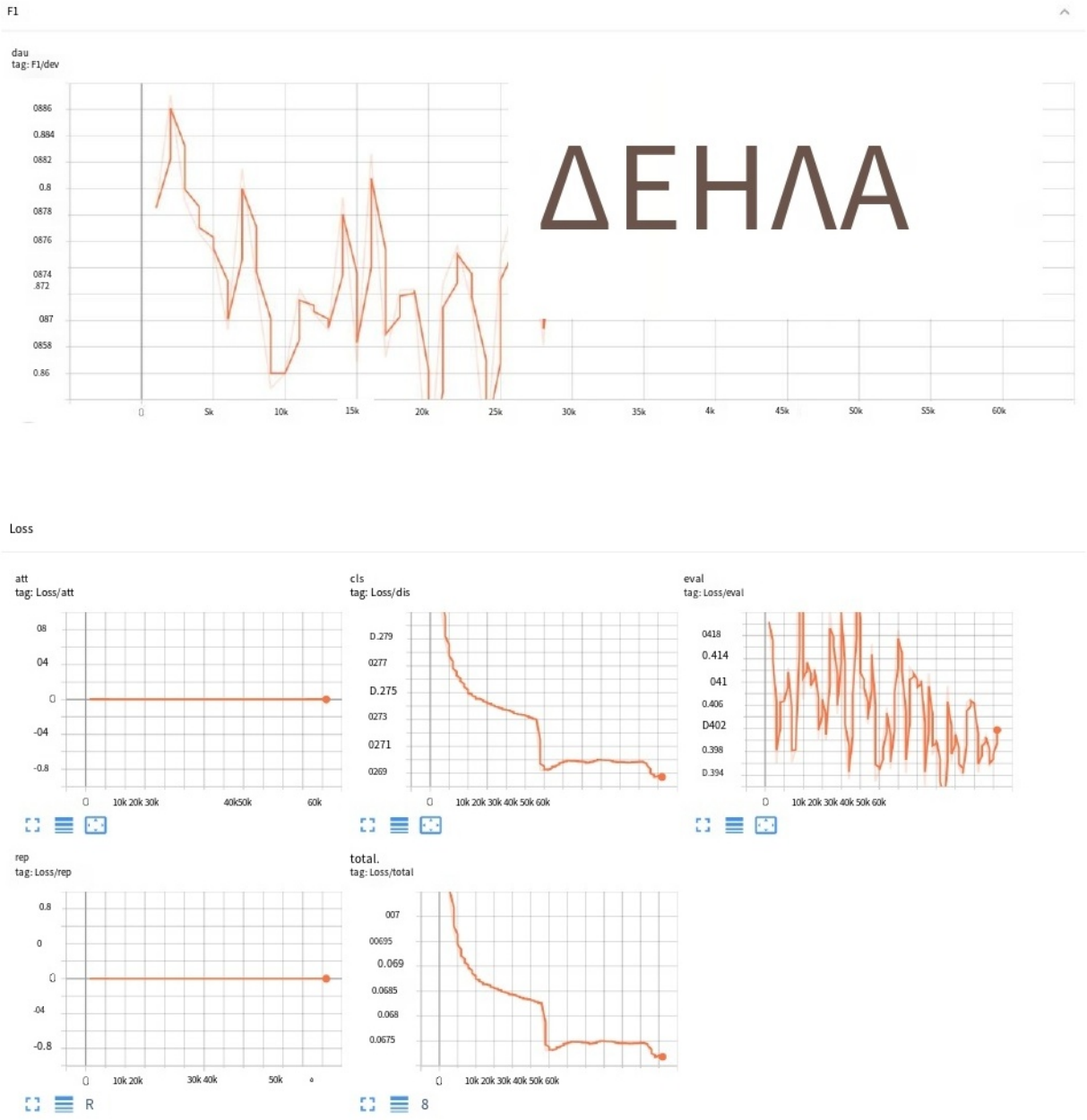
```
PS D:\subject\semester3.2\NLP> python tinybert\task_distill.py
>>       -pred_distill
>>     - teacher_model $env:FT_BERT_BASE_DIR
>>     - student_model $env:TMP_TINYBERT_DIR
>>     - data_dir $env:TASK_DIR
>>     - task_name MRPC
>>     -
       output_dir $env:TINYBERT_DIR aug_train
>>     -
>>     - do_lower_case
>>     - learning_rate 3e-5
>>     - num_train_epochs 3
>>     - eval_step 100
>>     - max_seq_length 128
>>     - train_batch_size 8
>>     - gradient_accumulation_steps 4
>>     - -eval_step 1000
```

The initial performance of each evaluation metric in the first round is as follows:



During the second and third training rounds, we observed a continuous decline in overall loss, with accuracy remaining around 0.82 and F1 score fluctuating around 0.86, meeting the reproducibility requirements. However, no significant fluctuations were noted as training progressed further. Considering this might be due to an issue with loss_eval, we decided to proceed with model improvements.





4.7 MRPC Distillation Improvement (Distillation for Specific Tasks)

Inspired by the related optimization paper [5], we attempt to modify the final layer's loss function into a hybrid loss combining cross-entropy from the teacher model and actual labels, with their weights dynamically adjusted during training. Initially, the loss is entirely based on the teacher model to enable rapid pattern learning; subsequently, the weight of the true label loss is gradually increased to achieve more stable and optimized accuracy evaluation.

Implement a dynamic weight adjustment strategy that starts with pure KD (knowledge distillation) loss and gradually introduces 0.5 CE (cross-entropy) loss over time. In the early training phase: use only KD loss (current_ratio = 0, i.e., cls_loss = 1.0 * kd_loss + 0.0 * ce_loss). In the later training phase: progressively increase the weight of CE loss until it reaches a final ratio of 0.5 KD + 0.5 CE (current_ratio = 0.5, i.e., cls_loss = 0.5 * kd_loss + 0.5 * ce_loss).

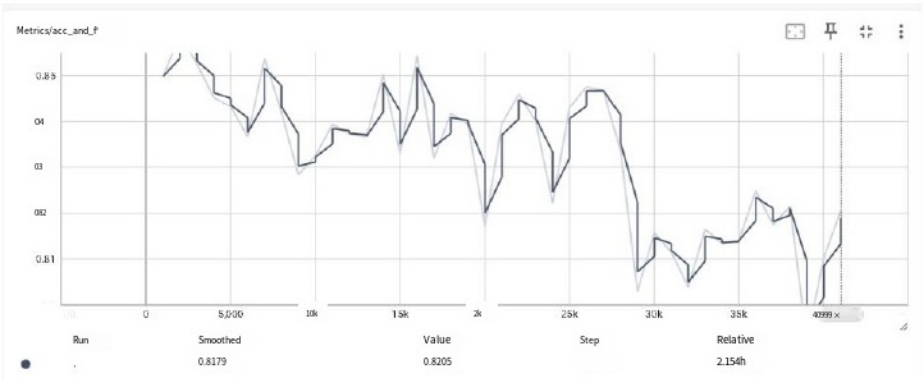
```
    else:
        if output_mode == "classification":
            kd_loss = soft_cross_entropy(student_logits / args.temperature,
                                         teacher_logits / args.temperature)
            ce_loss = CrossEntropyLoss()(student_logits.view(-1, num_labels),
                      label_ids.view(-1))
            current_ratio = min(0.5, 1 * (epoch_ / args.num_train_epochs))  #
            cls_loss = (1 - current_ratio) * kd_loss + current_ratio * ce_loss
            cls_loss = (1 - current_ratio) * kd_loss + current_ratio * ce_loss
        elif output_mode == "regression":
            loss_mse = MSELoss()
            cls_loss = loss_mse(student_logits.view(-1), label_ids.view(-1))

        loss = cls_loss
        tr_cls_loss += cls_loss.item()
```

```
PS D:\subject\semester3.2\NLP> $env:TINYBERT_DIR = "D:/subject/semester3.2/NLP/tinybert/final_tinybert_new/mrpc/"
PS D:\subject\semester3.2\NLP> python tinybert\task_distill_2.py
>>     --pred_distill
>>     -teacher_model $env:FT_BERT_BASE_DIR
>>     -student_model $env:TMP_TINYBERT_DIR
>>     -data_dir $env:TASK_DIR
>>     -task_name MRPC
>>     -output_dir $env:TINYBERT_DIR
>>     -aug_train
>>     -do_lower_case
>>     -learning_rate 3e-5
>>     -num_train_epochs 3
>>     -eval_step 100
>>     -max_seq_length 128
>>     --train_batch_size 8
>>     --gradient_accumulation_steps 4
>>     -eval step 100e
```

```
07/04 02:28:24 PM *** Running evaluation ***
Epoch = e iter 999 step
Num examples = 4e8
Batch size = 32
13/13 [00:00<00:00, 45.08 operations per second]
***** Eval results *****
acc = e.813725490196784
acc and f1 = e.8447712418309654
att_loss = 0.e
cls_loss = 0.3556885336262341
eval_loss = e.4421489949489778
f1 = e.8758169934648523
global_step = 999
loss = 6.888922133466558 52
rep_loss = e.e
10/13 [00:00<00:00, 44.89it/s]
Iteration: 4% = ###2
4788/112887 [82:54 < 1:82:26, 28.83 iterations per second]
```

Result analysis: The overall accuracy and F1 scores remain around 82%, likely due to limitations of the teacher fine-tuned model and insufficient dynamic adjustment of proportions.



Conclusion

TinyBERT achieves an efficient improvement of the BERT pre-trained language model through multi-layer distillation, featuring lightweight design, low memory usage, and low computational complexity, enabling efficient inference. It is highly user-friendly and can be quickly adapted to specific natural language processing tasks via fine-tuning. For domain-specific datasets, even with limited data volume, generalization and richness can be achieved through dual data augmentation using both BERT and GloVe.

Among various BERT distillation variants, it stands out for its high performance while fully preserving the evaluation advantages of the teacher model. It is suitable for a wide range of natural language processing tasks, including but not limited to text classification, question answering systems, and sentiment analysis. Due to its compact size, it is particularly well-suited for deployment on mobile devices or in low-power environments, such as smart assistants, chatbots, and real-time information filtering applications.

However, since the prediction layer's cls_loss is entirely based on consistency evaluation from the teacher model, TinyBERT's performance in specific domains largely depends on the teacher model. If the pre-trained teacher model does not perform well for a given domain, the student model struggles to surpass this upper limit. In this paper, we initially attempt a hybrid cls_loss formulation that dynamically combines teacher model consistency with actual label consistency, but it has not yet achieved ideal results. Further research is needed to optimize the dynamic weighting between these two components to further enhance model performance while maintaining lightweight design.

References

[1]. Sarzynska-Wawer J, Wawer A, Pawlak A, et al. Detecting formal thought disorder by deep contextualized word representations[J]. Psychiatry Research, 2021, 304: 114135

[2]. Achiam J, Adler S, Agarwal S, et al. Gpt-4 technical report[J]. arXiv preprint arXiv:2303.08774, 2023.

[3]. Devlin J, Chang M W, Lee K, et al. Bert: Pre-training of deep bidirectional transformers for language understanding[C]//Proceedings of the 2019 conference of the North American chapter of the association for computational linguistics: human language technologies, volume 1 (long and short papers). 2019: 4171-4186.

[4]. Jiao X, Yin Y, Shang L, et al. Tinybert: Distilling bert for natural language understanding[J]. arXiv preprint arXiv:1909.10351, 2019.

[5]. Chen X, He B, Hui K, et al. Simplified tinybert: Knowledge distillation for document retrieval[C]//Advances in Information Retrieval: 43rd European Conference on IR Research, ECIR 2021, Virtual Event, March 28−April 1, 2021, Proceedings, Part II 43. Springer International Publishing, 2021: 241-248.

[6]. Kovaleva O, Romanov A, Rogers A, et al. Revealing the dark secrets of BERT[J]. arXiv preprint arXiv:1908.08593, 2019.